

PROJECT DOCUMENTATION

Self-Healing Infrastructure Monitoring System

A DevOps-Based Autonomous Infrastructure Management Platform

Table of Contents

1. Project Overview	3
2. Problem Statement	3
3. Objectives	4
4. Technology Stack	4
5. System Architecture	5
6. Monitoring Pipeline — Data Flow	6
7. Project Folder Structure	7
8. Phase-by-Phase Implementation	7
8.1 Phase 1 — AWS Server Setup	7
8.2 Phase 2 — Prometheus + Grafana	8
8.3 Phase 3 — Alert System	9
8.4 Phase 4 — Self-Healing Scripts	10
8.5 Phase 5 — CI/CD Pipeline	12
9. Complete File Configurations	13
10. Self-Healing Logic Flow	17
11. CI/CD Pipeline Flow	18
12. Key Concepts & Terminology	19
13. Viva Preparation	20
14. References	21

1. Project Overview

The Self-Healing Infrastructure Monitoring System is a DevOps-based project that combines real-time server monitoring with autonomous fault recovery. Built on AWS EC2, the system continuously observes infrastructure health metrics — CPU, RAM, disk utilization, and container status — and automatically takes corrective action when anomalies are detected, without any human intervention.

This project demonstrates a production-grade DevOps workflow using industry-standard open-source tools including Prometheus for metrics collection, Grafana for visualization, Alertmanager for notifications, Docker for containerization, and GitHub Actions for continuous integration and deployment. A custom Python-based self-healing engine runs as a persistent background process, restarting crashed containers, cleaning overflowing disks, and logging every remediation action for audit purposes.

What makes this project unique in an academic context is its focus on operational automation rather than traditional machine learning prediction — it solves a real-world infrastructure problem that every cloud-based organization faces daily.

2. Problem Statement

Modern IT infrastructure demands continuous availability. Traditional monitoring approaches require manual intervention when systems encounter issues — an increasingly inadequate model in today's containerized, cloud-native environments. Specific problems this project addresses:

- Service outages go undetected until end users report failures, causing reputational and financial damage.
- Crashed Docker containers require manual identification and restart, leading to extended downtime.
- Disk overflow from unmanaged log accumulation causes application failures and data loss.
- High CPU or RAM utilization left unaddressed degrades performance across all hosted services.
- No centralized visibility into server health means administrators operate without situational awareness.
- Round-the-clock human monitoring is expensive, error-prone, and unsustainable at scale.

3. Objectives

- Build a real-time monitoring system tracking CPU, RAM, disk, and container health on AWS EC2.
- Implement self-healing scripts that automatically restart crashed containers within 30 seconds.
- Configure threshold-based alerting that sends email notifications via Gmail SMTP.
- Visualize all infrastructure metrics on a live Grafana dashboard accessible via browser.

- Integrate a GitHub Actions CI/CD pipeline for automated build and deployment on every code push.
 - Log all automated remediation actions to persistent audit log files.
 - Demonstrate a complete DevOps automation workflow applicable to real-world enterprise environments.
-

4. Technology Stack

Category	Tools / Technologies
Cloud Platform	Amazon Web Services (AWS) — EC2 t2.micro, Free Tier
Operating System	Ubuntu Server 22.04 LTS
Containerization	Docker CE, Docker Compose
Metrics Collection	Prometheus 2.x, Node Exporter
Visualization	Grafana (Dashboard ID: 1860)
Alerting	Alertmanager — Email via Gmail SMTP port 587
Self-Healing Engine	Python 3.x — threading, subprocess, shutil, logging
Shell Scripting	Bash — crontab, nohup, systemd
CI/CD Pipeline	GitHub Actions — appleboy/ssh-action
Version Control	Git, GitHub
Query Language	PromQL — Prometheus Query Language
Application Runtime	Python http.server (sample app on port 8080)

5. System Architecture

The system is composed of five interconnected layers deployed on a single AWS EC2 instance. Each layer serves a distinct role, and together they form a complete autonomous monitoring and remediation platform.

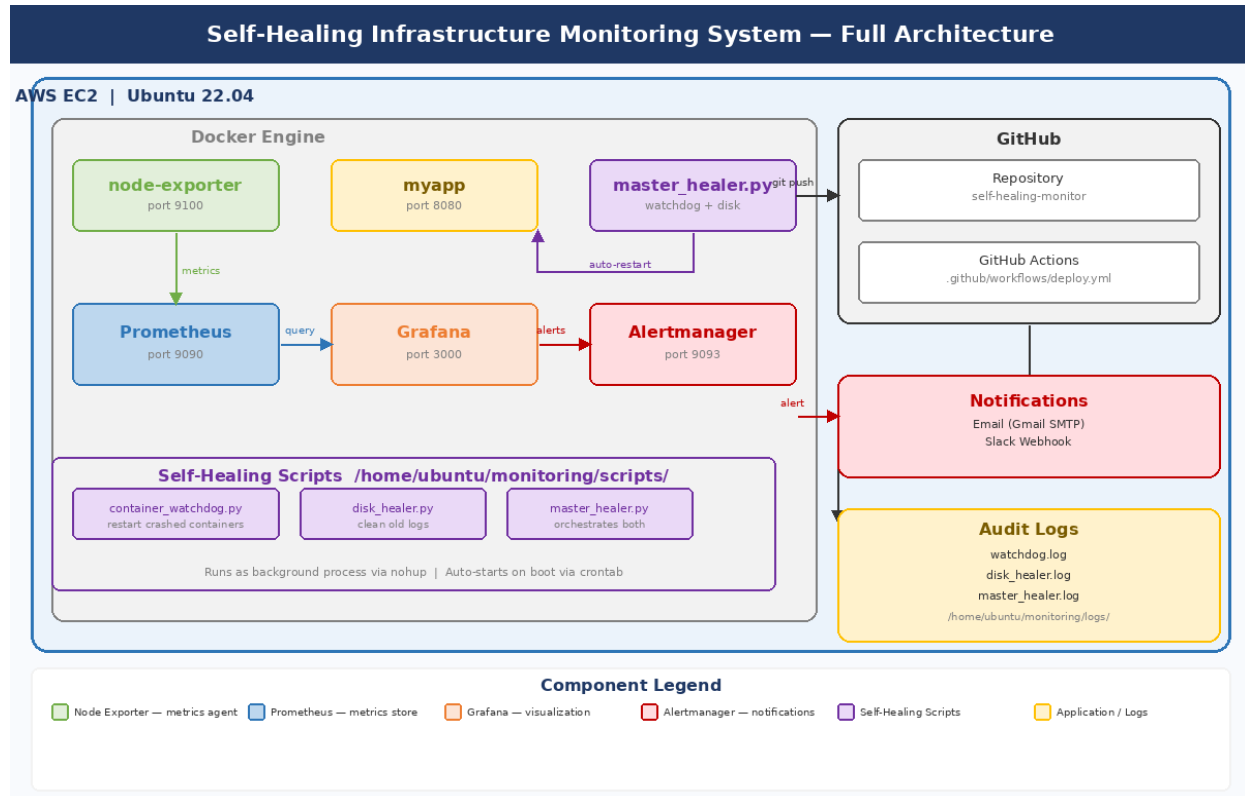


Figure 1 — Full System Architecture Diagram

5.1 Layer Overview

- **Metrics Collection Layer** — Node Exporter runs as a Docker container and exposes Linux system statistics at port 9100. Prometheus scrapes these metrics every 15 seconds.
- **Storage & Evaluation Layer** — Prometheus stores all time-series metrics locally and continuously evaluates alert rules defined in `alert.rules.yml` against the incoming data.
- **Visualization Layer** — Grafana connects to Prometheus as a data source and renders live dashboards. Dashboard ID 1860 (Node Exporter Full) provides comprehensive server health graphs.
- **Notification Layer** — Alertmanager receives fired alerts from Prometheus and routes them to configured receivers — in this case, Gmail via SMTP on port 587.
- **Self-Healing Layer** — `master_healer.py` runs as a persistent background process using Python threading. It monitors container health every 30 seconds and disk usage every 60 seconds, taking corrective action automatically.
- **CI/CD Layer** — GitHub Actions triggers on every push to the main branch, SSHs into the AWS server, and redeploys the application container automatically.

6. Monitoring Pipeline — Data Flow

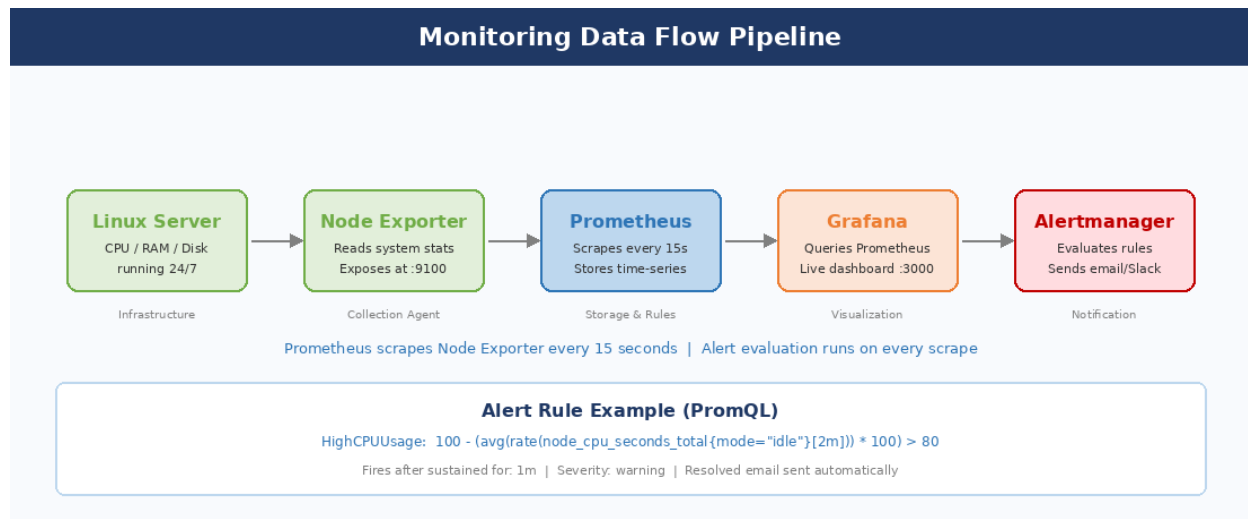


Figure 2 — Monitoring Data Flow from Server to Dashboard

6.1 Metric Collection Flow

Node Exporter reads Linux kernel statistics — from `/proc` and `/sys` filesystems — and exposes them as HTTP endpoints at port 9100. Prometheus scrapes this endpoint on a 15-second interval and stores the data as time-series. Grafana queries this data via PromQL to render graphs. When metric values breach defined thresholds, Prometheus fires an alert to Alertmanager, which dispatches an email notification.

6.2 Alert Rules (PromQL)

Alert Name	PromQL Expression	Threshold / Duration
HighCPUUsage	<code>100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100)</code>	> 80% for 1m
HighMemoryUsage	<code>1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100</code>	> 85% for 1m
HighDiskUsage	<code>node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes * 100</code>	> 90% for 1m
ContainerDown	<code>absent(container_last_seen{name="node-exporter"})</code>	for 30s

7. Project Folder Structure

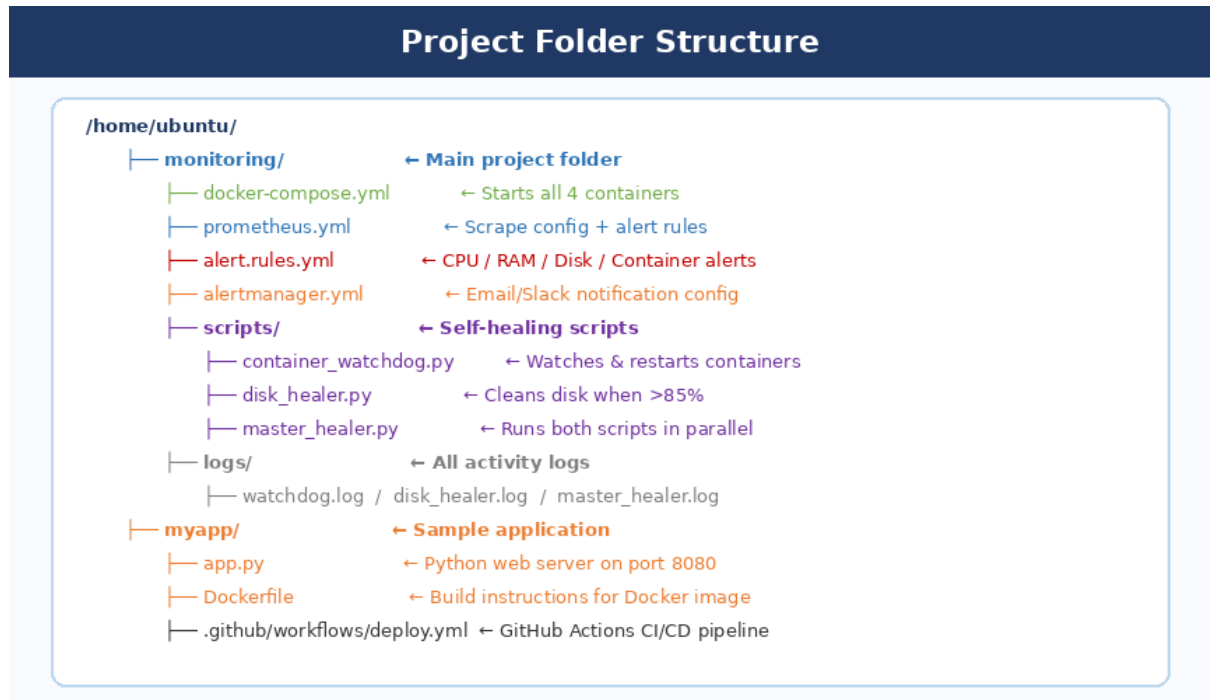


Figure 3 — Complete Project Folder Structure

8. Phase-by-Phase Implementation

8.1 Phase 1 — AWS Server Setup

An EC2 t2.micro instance running Ubuntu Server 22.04 LTS is launched on AWS Free Tier. The instance is configured with a security group allowing SSH (port 22), HTTP (port 80), HTTPS (port 443), and custom ports for monitoring tools (3000, 9090, 9093, 9100, 8080).

Key Commands

```
# Update server packages
sudo apt update && sudo apt upgrade -y

# Install Docker
sudo apt install docker.io -y
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker ubuntu && newgrp docker

# Install Docker Compose
sudo apt install docker-compose -y

# Verify installation
docker --version && docker-compose --version
```

8.2 Phase 2 — Prometheus + Grafana Setup

All monitoring components are orchestrated using Docker Compose. A single docker-compose.yml file defines four services: Prometheus (metrics storage), Node Exporter (metrics agent), Grafana (visualization), and Alertmanager (notifications). All services are configured with restart: always for basic Docker-level resilience.

prometheus.yml — Scrape Configuration

```
global:
  scrape_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']

rule_files:
  - 'alert.rules.yml'

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

docker-compose.yml

```
version: '3.8'
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - ./alert.rules.yml:/etc/prometheus/alert.rules.yml
    ports: ['9090:9090']
    restart: always

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports: ['9100:9100']
    restart: always

  alertmanager:
    image: prom/alertmanager:latest
    container_name: alertmanager
    volumes:
      - ./alertmanager.yml:/etc/alertmanager/alertmanager.yml
    ports: ['9093:9093']
    restart: always

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports: ['3000:3000']
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin123
    volumes:
      - grafana-data:/var/lib/grafana
    restart: always

volumes:
  grafana-data:
```

Access URLs After Deployment

Service	URL
---------	-----

Prometheus UI	<code>http://YOUR_IP:9090</code>
Grafana Dashboard	<code>http://YOUR_IP:3000 (admin / admin123)</code>
Alertmanager	<code>http://YOUR_IP:9093</code>
Node Exporter	<code>http://YOUR_IP:9100/metrics</code>
Sample App	<code>http://YOUR_IP:8080</code>

8.3 Phase 3 — Alert System

Alert rules are defined in PromQL and loaded into Prometheus. Alertmanager is configured to route all alerts to Gmail using SMTP authentication with a Google App Password (not the account password — required because Google blocks direct credential use for third-party apps).

alert.rules.yml

```
groups:
- name: server_alerts
  rules:
    - alert: HighCPUUsage
      expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100) > 80
      for: 1m
      labels:
        severity: warning
      annotations:
        summary: "High CPU Usage Detected"
        description: "CPU usage is above 80% for more than 1 minute."

    - alert: HighMemoryUsage
      expr: (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100 > 85
      for: 1m
      labels:
        severity: warning

    - alert: HighDiskUsage
      expr: (1 - (node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/" })) * 100 > 90
      for: 1m
      labels:
        severity: critical

    - alert: ContainerDown
      expr: absent(container_last_seen{name="node-exporter"})
      for: 30s
      labels:
        severity: critical
```

Testing Alerts — Stress Command

```
# Install stress tool
sudo apt install stress -y

# Spike CPU for 2 minutes — triggers HighCPUUsage alert
stress --cpu 4 --timeout 120

# Check firing alerts at:
# http://YOUR_IP:9090/alerts
```

8.4 Phase 4 — Self-Healing Scripts

The self-healing engine consists of three Python scripts. `master_healer.py` is the entry point — it spawns two daemon threads simultaneously using Python's threading module: one for container watchdog (checks every 30 seconds) and one for disk health (checks every 60 seconds).

master_healer.py — Complete Source

```
import threading, time, logging, subprocess, shutil, os
from datetime import datetime

logging.basicConfig(
```

```
filename='/home/ubuntu/monitoring/logs/master_healer.log',
level=logging.INFO,
format='%asctime)s - %(levelname)s - %(message)s'
)

MONITORED_CONTAINERS = ['prometheus','node-
exporter','alertmanager','grafana','myapp']
DISK_THRESHOLD = 85
LOG_PATHS = ['/var/log', '/tmp']

def get_running_containers():
    result = subprocess.run(['docker','ps','--format','{{.Names}}'],
                             capture_output=True, text=True)
    return result.stdout.strip().split('\n')

def restart_container(name):
    logging.warning(f"Container '{name}' DOWN. Restarting...")
    result = subprocess.run(['docker','restart',name], capture_output=True,
text=True)
    if result.returncode == 0:
        logging.info(f"'{name}' restarted successfully.")
    else:
        logging.error(f"Failed to restart '{name}': {result.stderr}")

def container_watchdog():
    while True:
        running = get_running_containers()
        for container in MONITORED_CONTAINERS:
            if container not in running:
                restart_container(container)
        time.sleep(30)

def get_disk_usage():
    total, used, free = shutil.disk_usage('/')
    return round((used / total) * 100, 2)

def clean_logs():
    for path in LOG_PATHS:
        for root, dirs, files in os.walk(path):
            for file in files:
                if file.endswith(('.log', '.gz', '.old')):
                    try:
                        os.remove(os.path.join(root, file))
                    except Exception as e:
                        logging.error(f'Error: {e}')

def disk_healer():
    while True:
        if get_disk_usage() > DISK_THRESHOLD:
            clean_logs()
        time.sleep(60)

if __name__ == '__main__':
    t1 = threading.Thread(target=container_watchdog, daemon=True)
    t2 = threading.Thread(target=disk_healer, daemon=True)
    t1.start(); t2.start()
    try:
        while True: time.sleep(1)
    except KeyboardInterrupt:
        logging.info('Master Healer stopped.')
```

Running the Healer Permanently

```
# Start in background (persists after terminal close)
nohup python3 ~/monitoring/scripts/master_healer.py \
  >> ~/monitoring/logs/master_healer.log 2>&1 &
```

```
# Auto-start on server reboot (add to crontab)
crontab -e
# Add this line:
@reboot sleep 30 && python3 /home/ubuntu/monitoring/scripts/master_healer.py \
    >> /home/ubuntu/monitoring/logs/master_healer.log 2>&1 &

# Verify it is running
ps aux | grep master_healer

# View live logs
tail -f ~/monitoring/logs/master_healer.log
```

8.5 Phase 5 — CI/CD Pipeline

A GitHub Actions workflow file automates the entire build and deployment lifecycle. On every git push to the main branch, GitHub spins up an Ubuntu runner, builds the Docker image, SSHs into the AWS server using stored secrets, and redeploys the application container. The monitoring system detects the new container instantly.

[.github/workflows/deploy.yml](#)

```
name: Build and Deploy

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Build Docker Image
        run: docker build -t self-healing-app:latest .

      - name: Deploy to Server via SSH
        uses: appleboy/ssh-action@master
        with:
          host: ${ secrets.SERVER_IP }
          username: ubuntu
          key: ${ secrets.SSH_PRIVATE_KEY }
          script: |
            cd ~/myapp
            git pull origin main
            docker stop myapp || true
            docker rm myapp || true
            docker build -t self-healing-app:latest .
            docker run -d --name myapp -p 8080:8080 self-healing-app:latest
            echo "Deployed at $(date)"
```

9. Complete File Configurations

9.1 alertmanager.yml

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 5m
  repeat_interval: 1h
  receiver: 'email-alert'

receivers:
- name: 'email-alert'
  email_configs:
  - to: 'YOUR_EMAIL@gmail.com'
    from: 'YOUR_EMAIL@gmail.com'
    smarthost: 'smtp.gmail.com:587'
    auth_username: 'YOUR_EMAIL@gmail.com'
    auth_identity: 'YOUR_EMAIL@gmail.com'
    auth_password: 'YOUR_APP_PASSWORD'
    send_resolved: true
```

9.2 app.py — Sample Application

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Self-Healing System - App is Running OK!')
    def log_message(self, format, *args):
        pass

print('App running on port 8080...')
HTTPServer(('0.0.0.0', 8080), Handler).serve_forever()
```

9.3 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
EXPOSE 8080
CMD ["python3", "app.py"]
```

10. Self-Healing Logic Flow

The following flowchart illustrates the complete decision logic of `master_healer.py` — showing both the container watchdog thread and the disk healer thread running in parallel as an infinite loop.

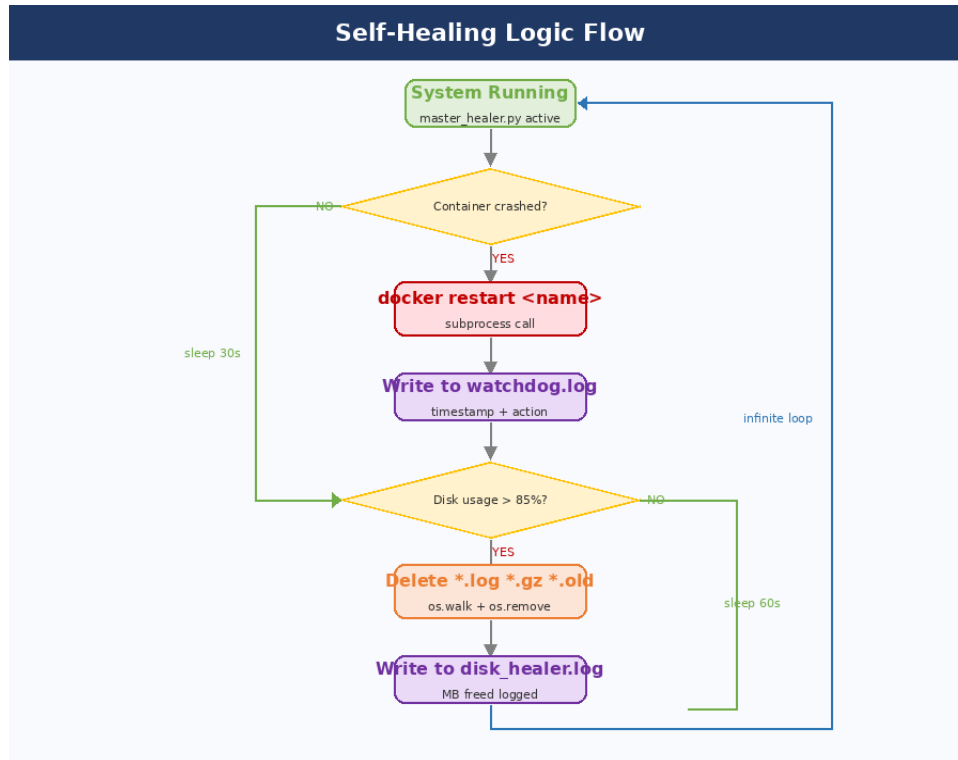


Figure 4 — Self-Healing Logic Flowchart

10.1 Container Recovery Sequence

1. `master_healer.py` starts on server boot via crontab @reboot
2. `container_watchdog` thread calls `docker ps` every 30 seconds
3. If a monitored container is absent from the running list, `restart_container()` is called
4. `subprocess.run(['docker', 'restart', name])` executes the Docker restart command
5. Return code 0 = success, logged to `watchdog.log` with timestamp
6. Prometheus detects container recovery and Alertmanager sends 'resolved' email

11. CI/CD Pipeline Flow

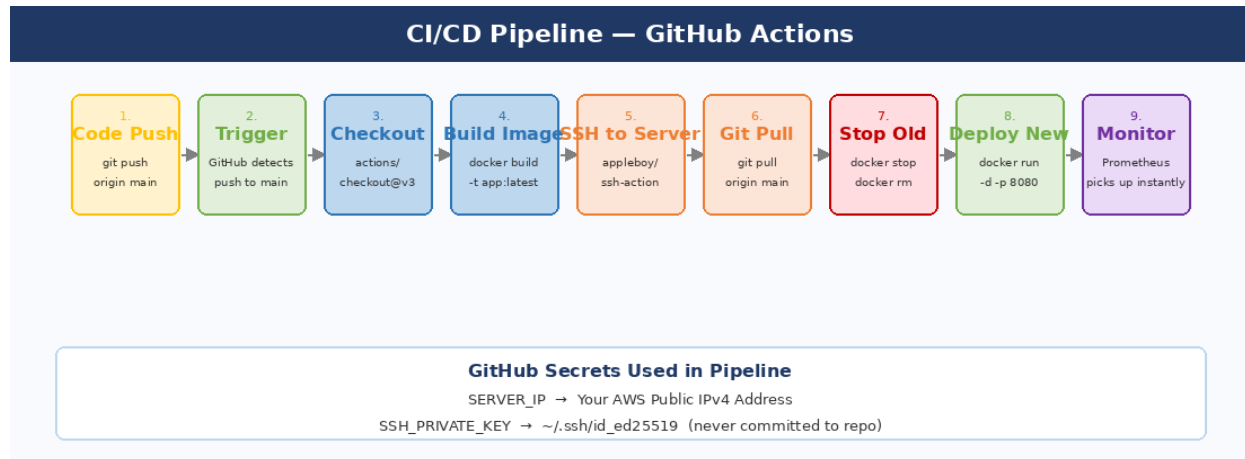


Figure 5 — GitHub Actions CI/CD Pipeline Stages

11.1 Pipeline Execution Steps

- Developer runs: git push origin main on local machine
- GitHub detects push event on main branch and triggers workflow
- GitHub spins up a fresh Ubuntu 22.04 runner (free with GitHub Actions)
- actions/checkout@v3 downloads repository code onto the runner
- docker build validates the Dockerfile builds successfully — acts as build test
- appleboy/ssh-action SSHs into AWS server using SERVER_IP and SSH_PRIVATE_KEY secrets
- git pull fetches latest code onto the production server
- docker stop and docker rm remove the old running container gracefully
- docker build creates a fresh image with the new code
- docker run starts the new container — application is live at port 8080
- Prometheus detects the new container and Grafana dashboard updates automatically

12. Key Concepts & Terminology

Term	Explanation
Docker	Platform for running applications in lightweight isolated containers
Container	An isolated, portable environment for running a single application process
Docker Compose	Tool for defining and running multi-container apps with a single YAML file
Prometheus	Open-source monitoring system that collects and stores time-series metrics
Node Exporter	Prometheus agent that reads Linux system stats from /proc and /sys
PromQL	Prometheus Query Language used to write alert rules and dashboard queries
Grafana	Visualization platform that renders Prometheus data as live graphs
Alertmanager	Handles alert routing, grouping, and delivery to email/Slack
SMTP	Simple Mail Transfer Protocol — used to send emails on port 587
Self-Healing	System automatically detects and fixes problems without human intervention
Threading	Python technique for running two tasks simultaneously in the same process
subprocess	Python module that runs shell commands (like docker restart) from Python code
nohup	Linux command that keeps a process running after the terminal is closed
crontab	Linux job scheduler — @reboot runs a command automatically on system boot
CI/CD	Continuous Integration/Deployment — automates build and deploy on code push
GitHub Actions	Free CI/CD service by GitHub that runs pipelines on their cloud runners
SSH Key Pair	Cryptographic key pair — private key authenticates to server without password
GitHub Secrets	Encrypted variables stored in GitHub — used to pass credentials to pipelines
EC2	AWS Elastic Compute Cloud — virtual servers in the Amazon cloud
Security Group	AWS firewall rules controlling which ports accept inbound traffic
restart: always	Docker Compose directive to auto-restart a container if it stops or crashes
daemon=True	Python thread flag — thread dies automatically when main program exits

13. Viva Preparation

13.1 Recommended Opening Statement

"I built a Self-Healing Infrastructure Monitoring System on AWS. The system has three layers — monitoring, alerting, and auto-remediation. Prometheus collects server metrics every 15 seconds. When CPU, RAM, or disk crosses a defined threshold, Alertmanager sends an email alert. The unique part is a Python script that runs 24/7 — if any Docker container crashes, it detects it within 30 seconds and restarts it automatically. Everything is visible on a live Grafana dashboard, and code changes deploy automatically through a GitHub Actions CI/CD pipeline."

13.2 Expected Viva Questions & Answers

Q: Why did you choose this project over an ML prediction system?

A: 95% of students build ML prediction systems. This is a real DevOps automation project that solves a genuine infrastructure problem. It demonstrates skills in Docker, Linux, Python, cloud, and CI/CD — all directly applicable to industry roles.

Q: What happens when a container crashes?

A: master_healer.py runs in the background and calls docker ps every 30 seconds. If a container is missing from the running list, it executes docker restart <container_name> via Python's subprocess module, logs the action, and the container is back up within seconds.

Q: What is PromQL and why do you use it?

A: PromQL is Prometheus Query Language. I use it to write alert rules. For example, the CPU alert uses: 100 minus the average idle CPU rate over 2 minutes. When this value exceeds 80% for 1 minute, Prometheus fires the alert to Alertmanager.

Q: How does the CI/CD pipeline work?

A: When I push code to GitHub, GitHub Actions triggers automatically. It checks out the code, builds a Docker image to verify it compiles, then SSHs into my AWS server using stored secrets, pulls the latest code, stops the old container, and starts a new one. The whole process takes under 2 minutes.

Q: Why do you use threading in master_healer.py?

A: The container watchdog runs every 30 seconds and the disk healer runs every 60 seconds. If I ran them sequentially, one would block the other. Using Python threading, both run simultaneously as daemon threads — fully parallel and non-blocking.

Q: What is the difference between Docker restart: always and your Python script?

A: restart: always is Docker's built-in restart policy — it only works if Docker daemon itself is running. My Python watchdog is an independent layer that can also detect degraded states, log all actions with timestamps, send notifications, and is extensible for custom logic that Docker cannot provide.

Q: How do you ensure the healer itself runs after a server reboot?

A: I added an @reboot entry to crontab that starts master_healer.py 30 seconds after boot (the delay ensures Docker is fully initialized before the script tries to check containers).

14. References

18. Prometheus Documentation. (2024). prometheus.io/docs — Configuration, PromQL, Alerting
19. Grafana Documentation. (2024). grafana.com/docs — Dashboard setup, Data sources
20. Docker Official Documentation. (2024). docs.docker.com — Compose, Networking
21. GitHub Actions Documentation. (2024). docs.github.com/en/actions — Workflows, Secrets
22. Amazon Web Services. (2024). docs.aws.amazon.com — EC2, Security Groups
23. Python Software Foundation. (2024). docs.python.org — threading, subprocess, shutil modules
24. Kim, G., Humble, J., Debois, P. & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.
25. Burns, B., Beda, J. & Hightower, K. (2019). Kubernetes: Up and Running. O'Reilly Media.
26. Node Exporter Full Dashboard. (2024). grafana.com/grafana/dashboards/1860

— End of Documentation —

Sachin Manoj Sharma | SMSC2526179 | MSc IT | 2025–2026